# VeriData — Agentic Data Scientist (Phase 1 Deliverable & Guide)

**Course Deliverable:** Phase 1 — Data Ingestion, Semantic Profiling, Deterministic Data Cleaning, Exploratory Data Analysis (EDA), Visualization, and Insights **Authors:** Maaz Mahmood (Partner 1: Core Data Engine, Profiling, 12-Step Cleaner, Scaling, Synthetic Benchmark) & Tahreem Zameer (Partner 2: Automated EDA Suite, Statistical Visualizations, Rule-Based Insights, Report Rendering)

---

## 1. Executive Summary: What This Project Is

**VeriData** is an **Agentic Data Scientist** system designed to automate rigorous data science workflows from raw CSV upload to statistically validated findings. Across its 5-phase roadmap (Phase 1: Ingestion, Cleaning & EDA; Phase 2: Hypothesis Generation; Phase 3: Statistical Testing & Benjamini-Hochberg Validation; Phase 4: LangGraph Agent & Chat; Phase 5: Benchmark Evaluation), the foundational rule of the system is:

> ***Every statistic, cleaning action, chart, and numeric claim is computed by deterministic, unit-tested Python code — never guessed or hallucinated by an LLM.***

**Phase 1** serves two roles simultaneously:

- **Role 1 — A Reusable, Deterministic Python Library & CLI (`src/ads/`):** Accepts *any* arbitrary CSV file, repairs encoding and delimiter quirks, infers semantic column types, profiles distributions and data quality defects, executes a conservative 12-step cleaning pipeline with cell-level audit logs, evaluates normalization/scaling transformations, generates 150-DPI visualizations, and writes versioned JSON/Markdown/HTML artifacts (`schema_version = "1.0"`) for Phase 2.
- **Role 2 — A Comprehensive Multi-Dataset Benchmark Study:** Evaluates the pipeline across **5 real-world public datasets** (`insurance`, `telco_churn`, `heart_disease`, `student_performance`, `bank_marketing`) plus **1 contamination-free synthetic messy benchmark** (`messy_synthetic`) injected with 18 categories of ground-truth data defects.

---

## 2. TA Submission Map: Exact Location of Every Required Deliverable

To make grading fast and transparent, the table below maps every requirement from the TA submission rubric to its exact file paths in this repository:

| # | TA Rubric Requirement | Exact Location in Repository | Notes for the Reviewer |
|---|---|---|---|
| 1 | **`README.md`** (title, description, problem statement, dataset details, cleaning summary, key findings, how to run notebooks) | `README.md` (repository root) | Contains dedicated sections for Problem Statement, Dataset Details, 12 Cleaning Steps, Key Findings per dataset, and Notebook instructions. |
| 2 | **Dataset Download Links & Cards** | `README.md` (Section 2), `docs/datasets.md`, `scripts/download_datasets.py` | Direct URLs (Kaggle, UCI, GitHub mirrors) are listed in `README.md` and `docs/datasets.md`. Running `python scripts/download_datasets.py --datasets all` downloads all 5 real datasets automatically. |
| 3 | **Jupyter Notebooks (`.ipynb`)** for cleaning, EDA & preprocessing with Markdown explanations and **visible outputs** | `notebooks/00_overview.ipynb` and `notebooks/<dataset>_phase1.ipynb` (7 notebooks total) | **All 7 notebooks have been pre-executed** so all tables, cleaning logs, scaling reports, and embedded PNG charts are immediately visible on GitHub or in VS Code / Jupyter without re-running cells. |
| 4 | **Graphs & Visualizations** (histograms, boxplots, correlation heatmaps, Cramér's V, missingness, target breakdowns) + **notes** | **1) Standalone PNGs:** `outputs/<dataset>/figures/*.png` **2) Inside Notebooks:** Section 4 of `notebooks/<dataset>_phase1.ipynb` **3) Reports:** `outputs/<dataset>/report.html` & `report.md` | Every chart is saved at 150 DPI in `outputs/<dataset>/figures/` and paired with a caption note explaining what the visualization reveals about the data. |
| 5 | **Cleaned / Processed Datasets** + scripts that generate them | **Cleaned CSVs (18 files):** `data/cleaned/<dataset>_cleaned.csv`, `<dataset>_cleaned_imputed.csv`, `<dataset>_scaled.csv` **Scripts:** `scripts/run_all.py`, `src/ads/pipeline.py` | Both conservative (`NaN` preserved) and imputed (median/mode filled with `__was_missing` flags) cleaned CSVs are committed in `data/cleaned/` for all 6 datasets. |

| | | | |
|---|---|---|---|
| 6 | **`requirements.txt`** listing all libraries | `requirements.txt` and `pyproject.toml` | Lists all pinned/compatible libraries grouped by subsystem (`pandas`, `numpy`, `scipy`, `scikit-learn`, `pydantic`, `pyyaml`, `matplotlib`, `seaborn`, `jinja2`, `jupyter`, `pytest`, etc.). |

---

## 3. End-to-End Pipeline Architecture

When `ads run --input <csv> --name <dataset>` (or `python scripts/run_all.py`) is executed, `ads.pipeline.run_phase1()` runs the following **8 deterministic stages** in strict sequence:

### Stage 1: Fault-Tolerant Two-Pass CSV Ingestion (`src/ads/ingestion.py`)

- **Problem Solved:** Calling `pd.read_csv()` with default arguments silently corrupts data (e.g., coercing Telco's `TotalCharges` to `object` because of 11 `" "` cells, failing on `;`-delimited European CSVs like `student_performance` and `bank_marketing`, or crashing on Latin-1 encodings and ragged rows).
- **How It Works:**
- Computes the raw file's **SHA-256 checksum** (`compute_sha256`) to guarantee raw data immutability.
- Probes file encoding in order (`utf-8` → `utf-8-sig` BOM → `latin-1`) and sniffs the delimiter (`,`, `;`, `tab`, `|`) over the first 64 KB.
- Normalizes header names (stripping whitespace/BOM, deduplicating repeated headers as `col.1`, renaming empty headers to `unnamed_<idx>`, and warning on exported index columns like `Unnamed: 0`).
- Loads **two aligned DataFrames**: `df_raw_str` (every cell preserved as an exact raw string with `dtype=str, keep_default_na=False`) and `df_naive` (for diagnostic comparison).

### Stage 2: 10-Class Semantic Type Inference (`src/ads/type_inference.py`)

- **Problem Solved:** Pandas physical dtypes (`object`, `int64`, `float64`) do not tell an analyst whether a column is a unique identifier (`customerID`), a binary flag (`Yes/No`, `0/1`), an integer-coded medical category (`cp` in Heart Disease), a currency-formatted number (`"$1,234.50"`), or a date.
- **Priority Cascade on Non-Missing Values:**
- **1. Config Override (`forced_types`):** Honors explicit domain overrides in `configs/datasets/<name>.yaml`.
- **2. `constant`:** ≤ 1 distinct non-missing value.
- **3. `id_like`:** 100% unique values (N > 50) matching ID/UUID/key naming patterns, uniform alphanumeric codes, or contiguous 0/1-based step-1 row indices.
- **4. `binary`:** Exactly 2 distinct non-null values (`0/1`, `Yes/No`, `True/False`, `M/F`).
- **5. `numeric_continuous` / `numeric_discrete`:** ≥ 95% of non-null tokens parse as numbers after stripping `$`, `,`, `%`, and accounting parentheses `(123)`. Integer columns with ≤ 15 unique values are classified as `numeric_discrete`; all others as `numeric_continuous`.
- **6. `datetime`:** ≥ 90% of non-null tokens parse as valid dates; mixed slash formats (`DD/MM` vs `MM/DD`) are automatically flagged with a warning.
- **7. `categorical_ordinal` / `categorical_nominal`:** Low-to-moderate cardinality string columns (matching known ordinal scales like `low/medium/high` or nominal categories).
- **8. `free_text` / `unknown`:** High-uniqueness long strings (median length > 30 chars) are marked `free_text` and excluded from statistical group comparisons.

### Stage 3: Dataset Profiling & Phase 2 Schema (`src/ads/profiling.py`)

- Computes `DataProfile` (`outputs/<dataset>/profile.json`): row/column counts, memory footprint, normalized duplicate row count, missingness matrix summary, and per-column statistics (`mean`, `std`, `median`, `q1`, `q3`, `iqr`, `skewness`, `kurtosis`, `n_zeros`, `n_negative`, 1.5×IQR / 3.0×IQR / Modified Z-score outlier counts, top 10 values, and string-quality flags).
- Writes `outputs/<dataset>/column_schema.json`, mapping each surviving column to its `semantic_type` and `role` (`feature`, `target_candidate`, `id`, `excluded`) for Phase 2.

### Stage 4: 12-Step Deterministic Cleaning Engine (`src/ads/cleaning/`)

Executes 12 ordered steps (`steps.py`, `categories.py`, `missing.py`, `outliers.py`, `log.py`):

- **Step 1 — Header Normalization:** Strips header whitespace/BOM and deduplicates column names.
- **Step 2 — Cell Whitespace & Unicode Cleanup:** Strips outer spaces, collapses internal multi-spaces, and removes zero-width Unicode characters (, ).

- **Step 3 — Missing Token Normalization:** Converts `""`, `"na"`, `"n/a"`, `"null"`, `"none"`, `"?"`, `"--"`, `"-999"`, `"missing"` (and `"unknown"` when configured) to `np.nan`.
- **Step 4 — Category Standardization:** Merges case/punctuation variants (`"North"` / `"north"` → majority form) and logs near-duplicate fuzzy typos (similarity ≥ 0.88, e.g., `"Nroth"`) for human review without destructive over-merging.
- **Step 5 — Type Conversion:** Converts formatted numeric strings (`"$1,234.50"`, `"15%"`, `"(45.00)"`) to `float64` / nullable `Int64`, and dates to ISO `YYYY-MM-DD`. Unparseable residual tokens become `NaN` (logged).
- **Step 6 — Domain Validity Enforcement:** Applies per-column `min`/`max`/allowed-set rules from `configs/datasets/<name>.yaml` (e.g., `age in [0, 120]`), setting impossible values to `NaN` (logged).
- **Step 7 — Duplicate Row Removal:** Drops exact duplicate rows under `drop_duplicates: auto` while keeping the first occurrence (`1` removed in `insurance`, `1` in `heart_disease`, `12` in `bank_marketing`, `40` in `messy_synthetic`).
- **Step 8 — Structural Drops:** Drops 100% empty rows, 100% `NaN` columns (`all_null_col`), and exported index columns (`Unnamed: 0`).
- **Step 9 — Dual Missing-Value Output Policy:**
- **Primary Conservative Output (`data/cleaned/<name>_cleaned.csv`):** Keeps true `NaN` intact so Phase 3 hypothesis tests are never biased by synthetic imputed values.
- **Secondary Imputed Output (`data/cleaned/<name>_cleaned_imputed.csv`):** Drops columns with > 80% missingness, imputes numeric columns with the **median** and categorical/binary columns with the **mode**, and appends `<col>__was_missing` (`0`/`1`) indicator columns.
- **Step 10 — Outlier Detection & Flagging:** Computes 1.5×IQR, 3.0×IQR, and Modified Z-scores (MAD = 0 guarded). Under the default `flag` policy, outliers are logged in `cleaning_log.json` and plotted in `outlier_summary.png` without deleting valid extreme records.
- **Step 11 — Scaling & Transformation Analysis (`src/ads/scaling.py`):** Evaluates skewness and scale disparity (max std / min std), recommending `log1p`, `yeo_johnson`, or `zscore` and writing `data/cleaned/<name>_scaled.csv`, `scaling_report.md`, and `scaling_params.json`.
- **Step 12 — Post-Condition & Idempotence Verification:** Verifies exact row accounting, zero remaining placeholder tokens, and idempotence (re-cleaning the conservative output produces zero further modifications).

### Stages 5–8: Automated EDA (`src/ads/eda/`), Insights (`src/ads/insights.py`), and Reports (`src/ads/reporting.py`)

- Generates **13–19 static PNG charts** per dataset (`outputs/<name>/figures/*.png`), markdown summary tables (`outputs/<name>/tables/*.md`), and `eda_summary.json`.
- Evaluates deterministic insight rules across 7 categories (`insights.json` and `insights.md`), preserving the analyst's manual domain notes (`<!-- MANUAL:START -->` ... `<!-- MANUAL:END -->`).
- Renders self-contained `outputs/<name>/report.md` and `outputs/<name>/report.html`.

---

## 4. Cross-Dataset Execution Summary

| Dataset | Raw Shape | Cleaned Shape | Imputed Cols | Raw Missing Cells | Duplicates | Logged Actions (Cells) | Figures | Insights |
|---|---|---|---|---|---|---|---|---|
| `insurance` | 1,338 × 7 | 1,337 × 7 | 7 | 0 (0.00%) | 1 | 3 (7) | 13 | 13 |
| `telco_churn` | 7,043 × 21 | 7,043 × 21 | 21 | 11 (0.01%) | 0 | 4 (22) | 16 | 17 |
| `heart_disease` | 303 × 14 | 302 × 14 | 14 | 0 (0.00%) | 1 | 6 (14) | 17 | 23 |
| `student_performance` | 395 × 33 | 395 × 33 | 33 | 0 (0.00%) | 0 | 11 (0) | 22 | 26 |
| `bank_marketing` | 41,188 × 21 | 41,176 × 21 | 22 | 12,718 (1.47%) | 12 | 22 (25,686) | 23 | 50 |
| `messy_synthetic` | 2,090 × 15 | 2,000 × 13 | 16 | 4,136 (13.19%) | 80 | 34 (12,214) | 19 | 37 |

**Semantic Type Inference Accuracy:** Across all **96 columns** of the 5 real datasets (`7` in `insurance` + `21` in `telco_churn` + `14` in `heart_disease` + `33` in `student_performance` + `21` in `bank_marketing`), our pipeline achieves **100.0% coarse semantic type accuracy** against the human-verified `truth_types` in `configs/datasets/*.yaml` (exceeding the ≥ 95% target in Section 18).

---

## 5. Dataset-by-Dataset Summary: Cleaning, Key Findings & File Locations

### 5.1 Medical Cost / Insurance (`insurance`)

- **Dataset Details:** `1,338` raw rows × `7` columns (`age`, `sex`, `bmi`, `children`, `smoker`, `region`, target `charges` in USD). Direct download: `https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/insurance.csv`
- **Cleaning Performed:** Removed `1` exact duplicate row (`1,338 → 1,337` rows); `0` missing values; flagged `139` rows (`10.40%`) in `charges` above the 1.5×IQR fence (`$34,524.78`).
- **Key Findings:**
- **Smoking status (`smoker`) dominates medical charges (eta² = 0.620):** Smokers average `$32,050` vs. `$8,441` for non-smokers.
- **Strong `smoker × bmi` interaction:** Obese smokers (BMI ≥ 30) form a distinct high-charge cluster (`$35,000–$63,770`), whereas BMI has only a mild slope among non-smokers.
- **Monotonic age effect:** `charges` rises consistently with `age` (Spearman rho = +0.534) across three parallel baseline tiers.
- **Scaling Decision:** `charges` is right-skewed (`skew = +1.52`); `log1p` transform is applied in `insurance_scaled.csv`.
- **Deliverable Locations:**
- **Notebook:** `notebooks/insurance_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/insurance_cleaned.csv`, `insurance_cleaned_imputed.csv`, `insurance_scaled.csv`
- **Report & Graphs:** `outputs/insurance/report.html` (`report.md`) · `outputs/insurance/figures/` (13 PNGs)

### 5.2 Telco Customer Churn (`telco_churn`)

- **Dataset Details:** `7,043` rows × `21` columns (`customerID`, demographics, `tenure`, `Contract`, internet/phone services, `MonthlyCharges`, `TotalCharges`, binary target `Churn`). Direct download: `https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv`
- **Cleaning Performed:** Detected `11` whitespace-only strings (`" "`) in `TotalCharges`, normalized them to `NaN` in conservative output, converted the remaining `7,032` values from `object` to `float64`, and median-imputed `TotalCharges` with `TotalCharges__was_missing` (`0/1`) in `telco_churn_cleaned_imputed.csv`. Configured `SeniorCitizen` (`0/1`) as categorical.
- **Key Findings:**
- **Structural Missingness (MAR):** All 11 missing `TotalCharges` cells occur where `tenure == 0` (new customers prior to first billing cycle).
- **Contract Term & Early-Tenure Churn:** `Contract` is the top categorical predictor of `Churn` (Cramér's V = 0.410; Month-to-month churn is `42.7%` vs. `2.8%` for Two-year contracts). `tenure` is strongly protective (Spearman rho = -0.353).
- **Collinearity & Shared Service Levels:** `TotalCharges` is strongly collinear with `tenure` (Pearson r = +0.826, Spearman rho = +0.890). Six internet add-on columns share the `"No internet service"` level (`21.7%` of rows).
- **Deliverable Locations:**
- **Notebook:** `notebooks/telco_churn_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/telco_churn_cleaned.csv`, `telco_churn_cleaned_imputed.csv`, `telco_churn_scaled.csv`
- **Report & Graphs:** `outputs/telco_churn/report.html` (`report.md`) · `outputs/telco_churn/figures/` (16 PNGs)

### 5.3 Heart Disease (`heart_disease`)

- **Dataset Details:** `303` raw rows × `14` columns (`age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca`, `thal`, binary target `target`). Direct download: `https://raw.githubusercontent.com/sharmaroshan/Heart-UCI-Dataset/master/heart.csv`
- **Cleaning Performed:** Removed `1` exact duplicate row (`303 → 302` rows); mapped integer-coded clinical categories (`cp`, `restecg`, `slope`, `thal`) to `categorical_nominal`; flagged `oldpeak` (`skew = +1.27`, `32.8%` zeros) and `chol` (`max = 564 mg/dL` outlier).
- **Key Findings:**
- **Top Categorical Predictors:** Thalassemia (`thal`, Cramér's V = 0.522), chest pain type (`cp`, Cramér's V = 0.508), and exercise-induced angina (`exang`, Cramér's V = 0.429).
- **Top Numeric Predictors:** Major vessels (`ca`, Spearman rho = -0.464), ST depression (`oldpeak`, Spearman rho = -0.434), and maximum heart rate (`thalach`, Spearman rho = +0.429).
- **Scaling Decision:** `oldpeak` (`skew = +1.27`) is transformed via `log1p` in `heart_disease_scaled.csv`.
- **Deliverable Locations:**
- **Notebook:** `notebooks/heart_disease_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/heart_disease_cleaned.csv`, `heart_disease_cleaned_imputed.csv`, `heart_disease_scaled.csv`

- **Report & Graphs:** `outputs/heart_disease/report.html` (`report.md`) · `outputs/heart_disease/figures/` (17 PNGs)

### 5.4 Student Performance — Math (`student_performance`)

- **Dataset Details:** `395` rows × `33` columns (`;`-delimited; demographics, family education, study time, `failures`, alcohol consumption, `absences`, period grades `G1`/`G2`, target final grade `G3` [`0`–`20`]). Direct download: `https://archive.ics.uci.edu/static/public/320/student+performance.zip`
- **Cleaning Performed:** Parsed `;`-delimited file, stripped quotes from string-quoted integer grades (`G1`, `G2`), verified `16` domain range rules; `0` missing cells and `0` duplicates.
- **Key Findings:**
- **Longitudinal Collinearity (`G1`, `G2` → `G3`):** `G2` (Spearman rho = +0.957, Pearson r = +0.905) and `G1` (rho = +0.879) almost completely determine `G3`.
- **Dropout Zero-Inflation (`G3 == 0`):** `38` students (`9.62%`) have `G3 = 0` with `0` recorded absences despite positive `G1`/`G2` grades, indicating exam non-attendance rather than continuous score variation.
- **Behavioral Predictors (Excluding `G1`/`G2`):** Past class `failures` (Spearman rho = -0.360, `78.7%` zeros) and mother's education (`Medu`, rho = +0.225) are the strongest non-grade predictors.
- **Deliverable Locations:**
- **Notebook:** `notebooks/student_performance_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/student_performance_cleaned.csv`, `student_performance_cleaned_imputed.csv`, `student_performance_scaled.csv`
- **Report & Graphs:** `outputs/student_performance/report.html` (`report.md`) · `outputs/student_performance/figures/` (22 PNGs)

### 5.5 Bank Marketing (`bank_marketing`)

- **Dataset Details:** `41,188` raw rows × `21` columns (`;`-delimited; client profile, campaign contact attributes, 5 macroeconomic indicators, binary target `y` [`yes`/`no`]). Direct download: `https://archive.ics.uci.edu/static/public/222/bank+marketing.zip`
- **Cleaning Performed:** Normalized **`12,718 "unknown"` placeholder tokens** across 6 columns (`default`: `8,597`, `education`: `1,731`, `housing`: `990`, `loan`: `990`, `job`: `330`, `marital`: `80`) to `NaN` in conservative output (and mode-imputed with `6` `<col>__was_missing` indicator columns in `bank_marketing_cleaned_imputed.csv`). Removed **`12` exact duplicate rows** (`41,188 → 41,176` rows).
- **Key Findings:**
- **Class Imbalance & Call Duration Leakage:** Target `y` has `11.27%` `yes` responses (`7.87 : 1` imbalance). Call `duration` has the strongest association with `y` (Spearman rho = +0.349), but is post-call target leakage for predictive modeling.
- **Macroeconomic Collinearity:** `euribor3m`, `emp.var.rate`, and `nr.employed` are nearly collinear (Pearson r = +0.94 to +0.97).
- **Sentinel Value & Block Missingness:** `pdays = 999` (`96.32%` of rows) denotes clients never previously contacted (eta² = 0.90 with `poutcome = nonexistent`), while missingness in `housing` and `loan` is 100% co-occurring (r = 1.00, same 990 rows).
- **Deliverable Locations:**
- **Notebook:** `notebooks/bank_marketing_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/bank_marketing_cleaned.csv`, `bank_marketing_cleaned_imputed.csv`, `bank_marketing_scaled.csv`
- **Report & Graphs:** `outputs/bank_marketing/report.html` (`report.md`) · `outputs/bank_marketing/figures/` (23 PNGs)

### 5.6 Messy Synthetic Benchmark (`messy_synthetic`)

- **Dataset Details:** Generated via `ads make-messy --seed 42` (`2,090` raw rows × `15` columns → `2,000` cleaned rows × `13` columns; binary target `churned`) with ground-truth answer key in `data/synthetic/messy_manifest.json`.
- **Cleaning Performed:** Dropped exported index (`Unnamed: 0`) and 100% null column (`all_null_col`); excluded `constant_col`; removed all injected duplicate rows (`2,090 → 2,000` rows); stripped whitespace/Unicode noise; normalized placeholder tokens; merged case variants in `region`; flagged fuzzy typo cells; parsed currency strings, percent strings, and mixed dates; and coerced out-of-range domain violations to `NaN` with **zero cell damage** on clean reference values.
- **Deliverable Locations:**
- **Notebook:** `notebooks/messy_synthetic_phase1.ipynb`
- **Cleaned CSVs:** `data/cleaned/messy_synthetic_cleaned.csv`, `messy_synthetic_cleaned_imputed.csv`, `messy_synthetic_scaled.csv`
- **Report & Graphs:** `outputs/messy_synthetic/report.html` (`report.md`) · `outputs/messy_synthetic/figures/` (19 PNGs)

## 6. How to Reproduce & Run Everything

- **Step 1 — Install dependencies and package:** `python -m pip install -r requirements.txt` followed by `python -m pip install -e .`
- **Step 2 — Download raw datasets & generate synthetic benchmark:** `python scripts/download_datasets.py --datasets all` and `python scripts/make_messy_dataset.py --seed 42 --variants`
- **Step 3 — Run Phase 1 pipeline on all 6 datasets:** `python scripts/run_all.py` (or single dataset via `python -m ads.cli run --input data/raw/insurance/insurance.csv --name insurance`)
- **Step 4 — Rebuild this master report (MD, HTML, and PDF):** `python scripts/build_report.py`
- **Step 5 — Open the pre-executed Jupyter notebooks:** `jupyter notebook notebooks/` (or open any `notebooks/<dataset>_phase1.ipynb` in VS Code / Cursor)